Online Complaint Register

Project Documentation

A MERN Stack Web Application for Complaint Registration and Management

Prepared by: Pandiri Santosh

1. Introduction

1.1 Project Title

Online Complaint Register

1.2 Team Members

Name	Role
Pandiri Santosh	Full Stack Developer (Frontend, Backend, Database, Documentation)

This is an individually developed project, with the sole contributor responsible for end-to-end design, development, testing, and deployment of the application.

2. Project Overview

2.1 Purpose

The Online Complaint Register is a web-based application that allows citizens to lodge complaints against civic issues — such as police, electricity, municipality, and water-related problems — and track their resolution in real time. The system connects three types of users: ordinary citizens who raise complaints, agents (officers) who are assigned to resolve specific complaints, and administrators who oversee the entire platform, assign work, and monitor system-wide activity.

The goal of the project is to digitize and streamline the traditional, often paper-based or in-person complaint process, replacing it with a transparent, trackable, and centrally managed digital workflow.

2.2 Key Features

User Registration & Profile Management

- Secure sign-up using email and password, with passwords hashed using bcrypt before storage.
- User profile stores name, email, phone number, and role (user, agent, or admin).

Complaint Lodging & Tracking

- Users can submit complaints by selecting a category (Police, Electricity, Municipality, Water, Other) and providing address, location, and a detailed description.
- Each complaint is automatically tagged with a status: Pending, In Review, or Resolved.
- Users can filter their own complaints by status and view agent notes/comments.

Messaging Thread

- A two-way message thread exists per complaint, allowing the citizen and the assigned agent to communicate directly within the context of that complaint.

Agent (Officer) Dashboard

- Agents view only the complaints assigned to them.
- Agents can update complaint status, add action notes describing resolution steps, and reply to citizen messages.

Admin Controls

- Admins can view all complaints across the system, filter by category or status, and assign complaints to specific agents.
- Admins can manage user accounts, including changing user roles (user/agent/admin) and removing accounts.
- A summary dashboard displays complaint counts by status (Total, Pending, In Review, Resolved) for quick monitoring.

Role-Based Authentication

- JWT (JSON Web Token) based authentication ensures each user only accesses functionality appropriate to their role.

3. Architecture

The application follows the MERN stack architecture (MongoDB, Express.js, React, Node.js) in a client-server model, where the React frontend acts as the client and the Express/Node.js backend acts as the server, communicating over RESTful APIs.

3.1 Frontend Architecture (React)

The frontend is built using React with Vite as the build tool, and is organized around three role-specific dashboards rendered conditionally based on the logged-in user's role.

- Component-based structure: Reusable components (Navbar, PrivateRoute) are separated from page-level components (Login, Register, UserDashboard, AgentDashboard, AdminDashboard).
- Routing: React Router DOM handles client-side navigation, with protected routes that redirect unauthenticated or unauthorized users back to the login page.
- State management: React's built-in useState and useEffect hooks manage local component state; a global AuthContext (built with React Context API) manages the logged-in user's identity and JWT token across the app.
- API communication: Axios is used to send HTTP requests to the backend, with the JWT token attached in the Authorization header for protected endpoints.
- Styling: Inline styles, Bootstrap, and Material UI (MUI) components are used together to create a clean, responsive interface for all three user roles.
- Persistence: The logged-in user's session (token and profile) is stored in the browser's localStorage so the user stays logged in across page refreshes.

3.2 Backend Architecture (Node.js + Express.js)

The backend exposes a RESTful API built with Express.js running on Node.js. It is organized into three layers: models, middleware, and routes.

- Models: Mongoose schemas define the structure of data stored in MongoDB (User, Complaint, Assigned, Message).
- Middleware: A custom authentication middleware verifies the JWT token on every protected request and attaches the decoded user information (id, name, role) to the request object.
- Routes: Each resource (auth, complaints, assigned, messages, users) has its own route file, keeping endpoint logic modular and maintainable.
- Security: Passwords are hashed using bcryptjs before being saved; sensitive routes check the requesting user's role before allowing access (role-based authorization).
- Cross-origin requests: The cors package allows the React frontend (running on a different port) to communicate with the backend API.

3.3 Database (MongoDB)

MongoDB stores all application data as JSON-like documents across four main collections, connected logically through ObjectId references (similar to foreign keys in relational databases).

Collection	Key Fields	Purpose
users	name, email, password, ph_no, user_type	Stores all registered accounts and their role (user / agent / admin)
complaints	user_id, category, address, city, state, pincode, description, status, comment	Stores each complaint lodged by a user, including its current resolution status
assigned	complaint_id, user_id, agent, status	Maps a complaint to the agent responsible for resolving it
messages	complaint_id, name, message	Stores the message thread exchanged between a user and an agent for a given complaint

Relationships: A user can have many complaints (one-to-many). A complaint can have one assignment record and many messages. The assigned collection links a complaint to the agent handling it, enabling the agent's dashboard to filter only relevant work.

4. Setup Instructions

4.1 Prerequisites

- Node.js (v18 or later) and npm
- MongoDB (local installation, accessed via MongoDB Compass) or a MongoDB Atlas cloud cluster
- Git, for cloning the repository
- A code editor such as Visual Studio Code

4.2 Installation Steps

1. Clone the repository:

```
git clone <repository-url>
cd online-complaint-register
```

2. Install backend dependencies:

```
cd backend
npm install
```

3. Create a .env file inside the backend folder with the following variables:

```
MONGO_URI=mongodb://localhost:27017/complaint_register
PORT=5000
JWT_SECRET=your_secret_key_here
```

4. Install frontend dependencies:

```
cd ../frontend
npm install
```

5. Ensure MongoDB is running locally (or update MONGO_URI to point to an Atlas cluster).

Once both installations complete successfully, proceed to the Running the Application section below.

5. Folder Structure

5.1 Client (Frontend) Structure

```
frontend/
  src/
    components/
      Navbar.jsx          - top navigation bar with role badge and logout
      PrivateRoute.jsx    - route guard for role-based access
    pages/
      Login.jsx           - login form
      Register.jsx        - registration form (user / agent)
      UserDashboard.jsx   - submit and track complaints, messaging
      AgentDashboard.jsx  - view assigned complaints, update status
      AdminDashboard.jsx  - manage complaints, assign agents, manage users
    context/
      AuthContext.jsx     - global authentication state
    App.jsx               - route definitions
    main.jsx              - React entry point
```

5.2 Server (Backend) Structure

```
backend/
  middleware/
    auth.js               - verifies JWT token on protected routes
  models/
    User.js                - user schema
    Complaint.js           - complaint schema
    Assigned.js             - assignment schema
```

```
  Message.js                - message schema
routes/
  auth.js                 - register and login endpoints
  complaints.js           - complaint CRUD endpoints
  assigned.js             - assignment endpoints
  messages.js             - messaging endpoints
  users.js                - admin user management endpoints
.env                      - environment variables (not committed to git)
server.js                 - Express app entry point
```

6. Running the Application

Both the backend and frontend servers must be running simultaneously, each in its own terminal.

6.1 Start the Backend Server

```
cd backend
npm start
```

The backend will run on http://localhost:5000 and will log "MongoDB connected" and "Server running on port 5000" once successfully started.

6.2 Start the Frontend Server

```
cd frontend
npm start
```

The frontend will run on http://localhost:5173 (Vite's default port). Open this URL in a browser to access the application.

Note: package.json scripts can be configured so that npm start runs node server.js on the backend and vite on the frontend, keeping startup commands consistent across both folders.

7. API Documentation

All endpoints are prefixed with /api. Protected endpoints require an Authorization header in the format: Bearer <token>.

7.1 Authentication Routes — /api/auth

Method	Endpoint	Description	Request Body
POST	/register	Register a new user or agent	name, email, password, ph_no, user_type
POST	/login	Authenticate and receive a JWT token	email, password

Example response for POST /login:

```
{
  "token": "eyJhbGciOi...",
  "user": {
    "id": "64f...",
    "name": "John Doe",
    "email": "john@email.com",
    "user_type": "user"
  }
}
```

7.2 Complaint Routes — /api/complaints

Method	Endpoint	Description	Access
POST	/	Submit a new complaint	Authenticated user
GET	/my	Get complaints submitted by the logged-in user	Authenticated user
GET	/	Get all complaints (optionally filtered by ?category=)	Agent / Admin
PATCH	/:id	Update a complaint's status or comment	Agent / Admin
DELETE	/:id	Delete a complaint	Admin only

Example response for GET /api/complaints/my:

```
[
  {
    "_id": "64f...",
    "user_id": "64a...",
    "category": "electricity",
    "address": "12 MG Road",
    "city": "Bengaluru",
    "state": "Karnataka",
    "pincode": "560001",
    "description": "No power since 2 days",
    "status": "pending",
    "comment": "",
    "createdAt": "2026-06-14T10:00:00.000Z"
  }
]
```

7.3 Assignment Routes — /api/assigned

Method	Endpoint	Description	Access
POST	/	Assign a complaint to an agent	Admin only

GET	/	Get assignments (agent sees only their own; admin sees all)	Agent / Admin
PATCH	/:id	Update an assignment's status	Agent / Admin

7.4 Message Routes — /api/messages

Method	Endpoint	Description	Access
POST	/	Send a message linked to a complaint	Authenticated user
GET	/:complaint_id	Get all messages for a complaint	Authenticated user

7.5 User Management Routes — /api/users

Method	Endpoint	Description	Access
GET	/	Get all registered users	Admin only
PATCH	/:id	Update a user's role	Admin only
DELETE	/:id	Delete a user account	Admin only

8. Authentication

The application uses JSON Web Tokens (JWT) for stateless authentication, rather than server-side sessions.

8.1 How It Works

6. When a user registers, their password is hashed using bcryptjs (10 salt rounds) before being stored in MongoDB — plain-text passwords are never saved.
7. When a user logs in, the backend compares the submitted password against the stored hash using bcrypt.compare().
8. On successful login, the backend generates a JWT signed with a secret key (JWT_SECRET), embedding the user's id, name, and user_type inside the token payload. The token expires after 1 day.
9. The frontend stores this token in localStorage and attaches it to every subsequent API request as an Authorization: Bearer <token> header.
10. A custom Express middleware (middleware/auth.js) intercepts protected routes, verifies the token's signature, and attaches the decoded payload to req.user, making the user's identity and role available to every route handler.

8.2 Authorization (Role-Based Access)

Beyond verifying identity, several routes additionally check req.user.user_type to enforce role-based permissions:

- Only admins can assign complaints to agents, delete complaints, or manage user accounts.
- Agents can only view and update assignments linked to their own user_id.
- Ordinary users can only view and act on their own complaints, never another user's data.

On the frontend, the PrivateRoute component reads the logged-in user's role from AuthContext and redirects unauthorized users away from dashboards that don't match their role, providing a second layer of protection in addition to the backend checks.

9. User Interface

The interface is organized into three distinct, role-specific views, each styled for clarity and ease of use.

9.1 Login & Registration

[Screenshot: Login page]

[Screenshot: Registration page]

9.2 User Dashboard

[Screenshot: Submit Complaint form]

[Screenshot: My Complaints list with status filters and messaging]

9.3 Agent Dashboard

[Screenshot: Assigned complaints list with status update and notes]

9.4 Admin Dashboard

[Screenshot: Complaint statistics overview]

[Screenshot: Complaint assignment to agents]

[Screenshot: User management panel]

10. Testing

Testing for this project was carried out primarily through manual functional testing across all three user roles, supplemented by direct inspection of API responses and database state.

10.1 Testing Approach

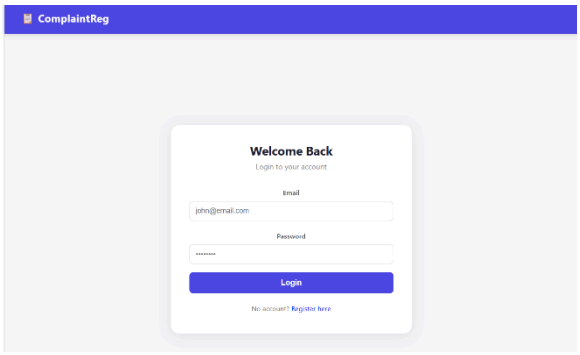
- Manual UI testing: Each feature (registration, login, complaint submission, status updates, messaging, agent assignment, user management) was tested directly through the browser across the User, Agent, and Admin dashboards.
- API testing: Endpoints were verified using Postman / browser dev tools to confirm correct status codes, response shapes, and role-based access restrictions (e.g. confirming a 403 response when a regular user attempts to access an admin-only route).
- Database verification: MongoDB Compass was used to visually confirm that documents were being created, updated, and linked correctly across the users, complaints, assigned, and messages collections.
- Authentication testing: Verified that protected routes reject requests with missing or invalid tokens, and that expired tokens force re-login.

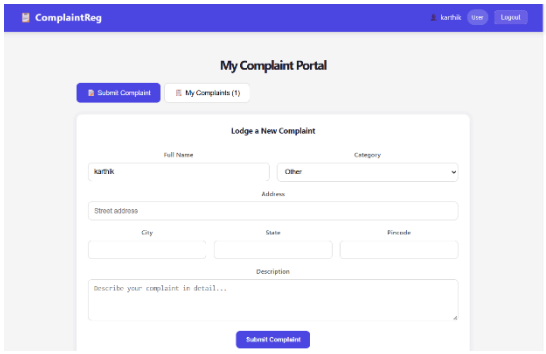
10.2 Tools Used

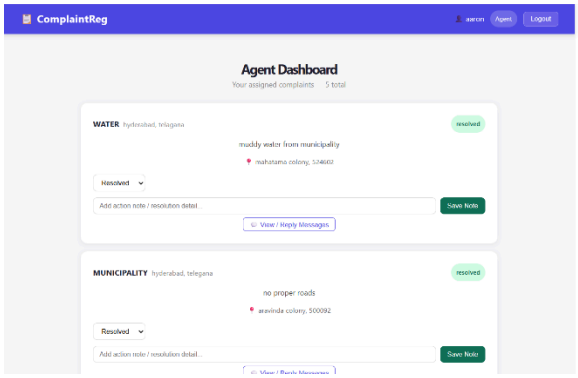
- Browser DevTools (Console & Network tabs) for debugging frontend requests and inspecting errors.
- MongoDB Compass for inspecting and editing collections directly during development.
- Postman (optional) for isolated backend endpoint testing.

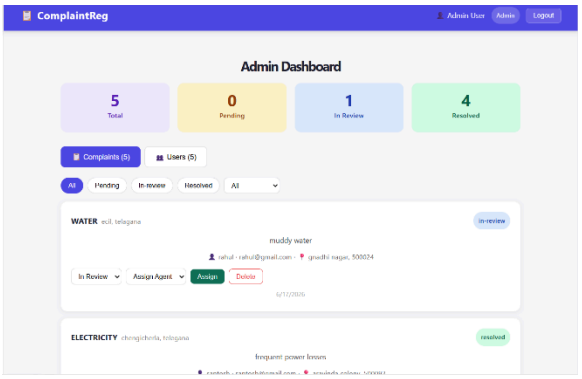
Future iterations of this project could introduce automated testing using Jest and Supertest for backend unit/integration tests, and React Testing Library for frontend component tests.

11. Screenshots or Demo









A complete walkthrough of the application is available as a recorded video demo, covering the following flow:

- User registration and login
- Submitting a complaint as a citizen
- Admin assigning the complaint to an agent
- Agent updating status and adding resolution notes
- User viewing the updated status and messaging the agent

Demo video:
https://drive.google.com/drive/folders/19zMvsoPEF1ue-3LJyjBphYIpMzGAIqxa?usp=sharing

12. Known Issues

- Duplicate assignment: An admin can currently assign the same complaint to an agent more than once, creating duplicate records in the assigned collection. A uniqueness check should be added to the assignment route.
- No email/SMS notifications: The features list calls for automated email/SMS updates on complaint submission and status changes; the current build only reflects status changes visually within the dashboard and does not yet send external notifications.
- No file/image attachments: Users cannot yet attach supporting documents or photos when lodging a complaint, despite this being a desired feature.
- No officer self-registration approval flow: Agents can currently register and log in immediately; the "admin approval before activation" step described in the requirements is not yet enforced.
- Admin account creation is manual: There is no in-app interface for creating the first admin account; it must currently be inserted directly into MongoDB or via a setup script.

- Limited input validation: Form fields (e.g. pincode, phone number) do not yet enforce strict format validation on the frontend or backend.

13. Future Enhancements

- Email and SMS notifications using a service such as Nodemailer or Twilio, triggered on complaint submission, status change, and resolution.
- File and image upload support for complaints, using a library such as Multer with cloud storage (e.g. AWS S3 or Cloudinary).
- Officer approval workflow, where newly registered agents remain inactive until an admin explicitly approves their account.
- Analytics dashboard enhancements, including charts (e.g. complaints by category, resolution time trends) using a charting library such as Chart.js or Recharts.
- Real-time updates using WebSockets (Socket.IO) so status changes and new messages appear instantly without requiring a page refresh.
- Search and advanced filtering, allowing admins and agents to search complaints by keyword, date range, or location.
- Automated testing suite covering backend routes (Jest + Supertest) and frontend components (React Testing Library).
- Deployment to a production environment (e.g. Render or Railway for the backend, Vercel or Netlify for the frontend, and MongoDB Atlas for the database).
- Audit logging to track every status change and assignment action for accountability and dispute resolution.